一、 实验一代码

```
#include <iostream>
#include <iomanip>
#include <string>
using namespace std;
int RULENUM = 15;//动物识别系统的规则库有条规则
int CAUSENUM = 5;//各条规则中的前提条件最多有个
int count1 = 0;//记录所选择的动物特征的个数
/*事实*/
string fact[] = { "", "有毛发", "有奶", "有羽毛", "会飞", "会下蛋", "吃肉", "有犬齿", "有爪",
    "眼盯前方", "有蹄", "嚼反刍", "黄褐色", "身上有暗斑点", "身上有黑色条纹", "有长脖子",
    "有长腿", "不会飞", "会游泳", "有黑白二色", "善飞", "哺乳动物", "鸟", "食肉动物", "蹄类动物",
    "金钱豹", "虎", "长颈鹿", "斑马", "鸵鸟", "企鹅", "信天翁" };
/*规则*/
int rule[][6] = { { 0, 0, 0, 0, 0, 0 }, { 21, 1, 0, 0, 0, 0 }, { 21, 2, 0, 0, 0, 0 }, { 22, 3,
    0, 0, 0, 0 }, { 22, 4, 5, 0, 0, 0 }, { 23, 6, 0, 0, 0, 0 }, { 23, 7, 8, 9, 0, 0 }, { 24,
    21, 10, 0, 0, 0 }, { 24, 21, 11, 0, 0, 0 }, { 25, 21, 23, 12, 13, 0 }, { 26, 21, 23, 12,
    14, 0 }, { 27, 24, 15, 16, 13, 0 }, { 28, 24, 14, 0, 0, 0 }, { 29, 22, 15, 16, 17, 19 }, {
    30, 22, 18, 17, 19, 0 }, { 31, 22, 20, 0, 0, 0 } };
int conditionlist[24];//所选择的动物特征
bool match(int a);       //函数声明
int inference();
int main()
{
   cout << "以下是一些动物的特征:" << endl;
   for (int i = 1; i <= 24; i++)
   {
      cout << i << ".";
      cout << setiosflags(ios::left) << setw(14) << fact[i] << " ";
      if (i % 4 == 0)
      {
         cout << endl;
      }
   }
   int a;
   int k = 0;
   cout << "请选择动物的特征:" << endl;
   while ((cin >> a))                //a为整形的，按任意字母可结束循环
   {
      conditionlist[k] = a;
      k++;
      count1++;
   }
   cout << "该动物名称为: " << endl;
   inference();
```

```
    system("pause");
    return 0;
}
/*知识匹配*/
bool match(int a)
{
    int i = 1;
    int j = 0;
    int flag = 0;
    int flag1 = 0;
    int num = 0;
    while ((flag == i - 1) && (flag1 != count1))
    {
        flag = 0;
        for (i = 1; i <= CAUSENUM; i++)
        {
            if (rule[a + num][i] == 0)
                break;
            for (j = 0; j < count1; j++)              //所选择的动物特征的条数
            {
                if (rule[a + num][i] == conditionlist[j])//匹配上一个条件flag计数加一
                {
                    flag++;
                    flag1++;
                    break;
                }
            }
        }
        num++;
    }
    if ((flag != i - 1) || (count1 == 0))
    {
        return false;
    }
    else
    {
        for (i = 0; i < num; i++)
        {
            for (j = 0; j < num; j++)
            {
                if ((i != j) && (rule[a + i][0] != rule[a + j][0]))
                {
                    return false;
                }
            }
        }
        return true;
```

```
    }
}
/*推理*/
int inference()
{
    int i;
    int j = 0;
    for (i = 1; i <= RULENUM; i++)
    {
        if (match(i) == true)
        {
            cout << fact[rule[i][0]] << endl;
            return 0;
        }
    }
    cout << "未知动物" << endl;
    getchar();
    return 0;
}
```

二、 实验二代码

```
#include<iostream>
#include<algorithm>
#include<string>
#include<vector>
#include<cmath>
#include<queue>
#define N 6 // 棋盘/迷宫 的阶数
using namespace std;

class Node
{
public:
   int x, y; // 节点所在位置
   int F, G, H; // G:从起点开始，沿着产的路径，移动到网格上指定方格的移动耗费。
   // H:从网格上那个方格移动到终点B的预估移动耗费，使用曼哈顿距离。
   // F = G + H
   Node(int a, int b) :x(a), y(b) {}

   // 重载操作符，使优先队列以F值大小为标准维持堆
   bool operator < (const Node& a) const
   {
      return F > a.F;
   }
};

// 定义八个方向
int dir[8][2] = { { -1, -1 }, { -1, 0 }, { -1, 1 }, { 0, -1 },
{ 0, 1 }, { 1, -1 }, { 1, 0 }, { 1, 1 } };
// 优先队列，就相当于open表
priority_queue<Node>que;
// 棋盘
int qp[N][N] = { { 0, 0, 0, 0, 0, 0 },
{ 0, 1, 1, 0, 1, 1 },
{ 0, 0, 1, 0, 0, 0 },
{ 0, 0, 1, 1, 1, 0 },
{ 0, 1, 1, 0, 0, 0 },
{ 1, 1, 0, 0, 0, 0 } };
bool visit[N][N]; // 访问情况记录，close表
int valF[N][N]; // 记录每个节点对应的F值
int path[N][N][2]; // 存储每个节点的父节点

int Manhuattan(int x, int y, int x1, int y1); // 计算曼哈顿距离
bool NodeIsLegal(int x, int y, int xx, int yy); // 判断位置合法性
void A_start(int x0, int y0, int x1, int y1); // A*算法
```

```
void PrintPath(int x1, int y1); // 打印路径

/* ----------------主函数------------------- */
int main()
{
   fill(visit[0], visit[0] + N * N, false); // 将visit数组赋初值false
   fill(valF[0], valF[0] + N * N, 0); // 初始化F全为0
   fill(path[0][0], path[0][0] + N * N * 2, -1); // 路径同样赋初值-1

   // // 起点 // 终点
   int x0, y0, x1, y1;
   cout << "输入起点: ";
   cin >> x0 >> y0;
   cout << "输入终点: ";
   cin >> x1 >> y1;
   x0--; y0--; x1--; y1--;

   if (!NodeIsLegal(x0, y0, x0, y0))
   {
      cout << "非法起点! " << endl;
      return 0;
   }

   A_start(x0, y0, x1, y1); // A*算法
   PrintPath(x1, y1);     // 打印路径
   system("pause");
   return 0;
}

/* ----------------自定义函数------------------ */
void A_start(int x0, int y0, int x1, int y1)
{
   // 初始化起点
   Node node(x0, y0);
   node.G = 0;
   node.H = Manhuattan(x0, y0, x1, y1);
   node.F = node.G + node.H;
   valF[x0][y0] = node.F;
   // 起点加入open表
   que.push(node);

   while (!que.empty())
   {
      Node node_top = que.top(); que.pop();
      visit[node_top.x][node_top.y] = true; // 访问该点, 加入closed表
      if (node_top.x == x1 && node_top.y == y1) // 到达终点
         break;
```

```
    // 遍历node_top周围的8个位置
    for (int i = 0; i < 8; i++)
    {
      Node node_next(node_top.x + dir[i][0], node_top.y + dir[i][1]); //
          创建一个node_top周围的节点
      // 该节点坐标合法 且 未加入close表
      if (NodeIsLegal(node_next.x, node_next.y, node_top.x, node_top.y) &&
          !visit[node_next.x][node_next.y])
      {
        // 计算从起点并经过node_top节点到达该节点所花费的代价
        node_next.G = node_top.G + int(sqrt(pow(dir[i][0], 2) + pow(dir[i][1], 2)) * 10);
        // 计算该节点到终点的曼哈顿距离
        node_next.H = Manhuattan(node_next.x, node_next.y, x1, y1);
        // 从起点经过node_top和该节点到达终点的估计代价
        node_next.F = node_next.G + node_next.H;

        // node_next.F < valF[node_next.x][node_next.y] 说明找到了更优的路径，则进行更新
        // valF[node_next.x][node_next.y] == 0 说明该节点还未加入open表中，则加入
        if (node_next.F < valF[node_next.x][node_next.y] || valF[node_next.x][node_next.y]
            == 0)
        {
          // 保存该节点的父节点
          path[node_next.x][node_next.y][0] = node_top.x;
          path[node_next.x][node_next.y][1] = node_top.y;
          valF[node_next.x][node_next.y] = node_next.F; // 修改该节点对应的valF值
          que.push(node_next); // 加入open表
        }
      }
    }
  }
}

void PrintPath(int x1, int y1)
{
  if (path[x1][y1][0] == -1 || path[x1][y1][1] == -1)
  {
    cout << "没有可行路径！" << endl;
    return;
  }
  int x = x1, y = y1;
  int a, b;
  while (x != -1 || y != -1)
  {
    qp[x][y] = 2; // 将可行路径上的节点赋值为2
    a = path[x][y][0];
    b = path[x][y][1];
```

```
      x = a;
      y = b;
    }
    // 表示未经过的节点，  表示障碍物，  表示可行节点
    string s[3] = { " ", " ", " " };
    for (int i = 0; i < N; i++)
    {
      for (int j = 0; j < N; j++)
        cout << s[qp[i][j]];
      cout << endl;
    }
}

int Manhuattan(int x, int y, int x1, int y1)
{
    return (abs(x - x1) + abs(y - y1)) * 10;
}

bool NodeIsLegal(int x, int y, int xx, int yy)
{
    if (x < 0 || x >= N || y < 0 || y >= N) return false; // 判断边界
    if (qp[x][y] == 1) return false; // 判断障碍物
    // 两节点成对角型且它们的公共相邻节点存在障碍物
    if (x != xx && y != yy && (qp[x][yy] == 1 || qp[xx][y] == 1)) return false;
    return true;
}
```

三、 实验三代码

```
%% 清空环境变量
clear all
clc

%% 训练集/测试集产生
load spectra_data.mat
% 随机产生训练集和测试集
temp = randperm(size(NIR,1));
% 训练集——50个样本
P_train = NIR(temp(1:50),:)';
T_train = octane(temp(1:50),:)';
% 测试集——10个样本
P_test = NIR(temp(51:end),:)';
T_test = octane(temp(51:end),:)';
N = size(P_test,2);

%% BP神经网络创建、训练及仿真测试(R2014a)

% 创建网络
net = feedforwardnet(9);
% 设置训练参数
net.trainParam.epochs = 1000;
net.trainParam.goal = 1e-3;
net.trainParam.lr = 0.01;
% 训练网络
net = train(net,P_train,T_train);
% 仿真测试
T_sim_bp = net(P_test);

%% RBF神经网络创建及仿真测试

% 创建网络
net = newrbe(P_train,T_train,0.3);
% 仿真测试
T_sim_rbf = sim(net,P_test);

%% 性能评价

% 相对误差error
error_bp = abs(T_sim_bp - T_test)./T_test;
error_rbf = abs(T_sim_rbf - T_test)./T_test;
% 决定系数R^2
R2_bp = (N * sum(T_sim_bp .* T_test) - sum(T_sim_bp) * sum(T_test))^2 / ((N *
    sum((T_sim_bp).^2) - (sum(T_sim_bp))^2) * (N * sum((T_test).^2) - (sum(T_test))^2));
```

```
R2_rbf = (N * sum(T_sim_rbf .* T_test) - sum(T_sim_rbf) * sum(T_test))^2 / ((N *
    sum((T_sim_rbf).^2) - (sum(T_sim_rbf))^2) * (N * sum((T_test).^2) - (sum(T_test))^2));
% 结果对比
result_bp = [T_test' T_sim_bp' T_sim_rbf' error_bp' error_rbf']

%% 绘图
figure
plot(1:N,T_test,'b:*',1:N,T_sim_bp,'r-o',1:N,T_sim_rbf,'k-.^')
legend('真实值','BP预测值','RBF预测值')
xlabel('预测样本')
ylabel('辛烷值')
string = {'测试集辛烷值含量预测结果对比(BP vs RBF)';['R^2=' num2str(R2_bp) '(BP)' ' R^2='
    num2str(R2_rbf) '(RBF)']};
title(string)
```

四、 实验四代码

```
%% 清空环境变量
clear all
clc

%% 导入数据
load citys_data.mat

%% 计算城市间相互距离
n = size(citys,1);
D = zeros(n,n);
for i = 1:n
    for j = 1:n
        if i ~= j
            D(i,j) = sqrt(sum((citys(i,:) - citys(j,:)).^2));
        else
            D(i,j) = 1e-4;
        end
    end
end

%% 初始化参数
m = 50;                          % 蚂蚁数量
alpha = 1;                       % 信息素重要程度因子
beta = 5;                        % 启发函数重要程度因子
rho = 0.1;                       % 信息素挥发因子
Q = 1;                           % 常系数
Eta = 1./D;                      % 启发函数
Tau = ones(n,n);                 % 信息素矩阵
Table = zeros(m,n);              % 路径记录表
iter = 1;                        % 迭代次数初值
iter_max = 200;                  % 最大迭代次数
Route_best = zeros(iter_max,n); % 各代最佳路径
Length_best = zeros(iter_max,1); % 各代最佳路径的长度
Length_ave = zeros(iter_max,1); % 各代路径的平均长度

%% 迭代寻找最佳路径
while iter <= iter_max
    % 随机产生各个蚂蚁的起点城市
      start = zeros(m,1);
      for i = 1:m
          temp = randperm(n);
          start(i) = temp(1);
      end
      Table(:,1) = start;
```

```
% 构建解空间
citys_index = 1:n;
% 逐个蚂蚁路径选择
for i = 1:m
    % 逐个城市路径选择
   for j = 2:n
       tabu = Table(i,1:(j - 1));      % 已访问的城市集合(禁忌表)
       allow_index = ~ismember(citys_index,tabu);
       allow = citys_index(allow_index); % 待访问的城市集合
       P = allow;
       % 计算城市间转移概率
       for k = 1:length(allow)
           P(k) = Tau(tabu(end),allow(k))^alpha * Eta(tabu(end),allow(k))^beta;
       end
       P = P/sum(P);
       % 轮盘赌法选择下一个访问城市
       Pc = cumsum(P);
      target_index = find(Pc >= rand);
      target = allow(target_index(1));
      Table(i,j) = target;
   end
end
% 计算各个蚂蚁的路径距离
Length = zeros(m,1);
for i = 1:m
    Route = Table(i,:);
    for j = 1:(n - 1)
        Length(i) = Length(i) + D(Route(j),Route(j + 1));
    end
    Length(i) = Length(i) + D(Route(n),Route(1));
end
% 计算最短路径距离及平均距离
if iter == 1
    [min_Length,min_index] = min(Length);
    Length_best(iter) = min_Length;
    Length_ave(iter) = mean(Length);
    Route_best(iter,:) = Table(min_index,:);
else
    [min_Length,min_index] = min(Length);
    Length_best(iter) = min(Length_best(iter - 1),min_Length);
    Length_ave(iter) = mean(Length);
    if Length_best(iter) == min_Length
        Route_best(iter,:) = Table(min_index,:);
    else
        Route_best(iter,:) = Route_best((iter-1),:);
    end
end
```

```
    % 更新信息素
    Delta_Tau = zeros(n,n);
    % 逐个蚂蚁计算
    for i = 1:m
        % 逐个城市计算
        for j = 1:(n - 1)
            Delta_Tau(Table(i,j),Table(i,j+1)) = Delta_Tau(Table(i,j),Table(i,j+1)) +
                Q/Length(i);
        end
        Delta_Tau(Table(i,n),Table(i,1)) = Delta_Tau(Table(i,n),Table(i,1)) + Q/Length(i);
    end
    Tau = (1-rho) * Tau + Delta_Tau;
  % 迭代次数加1，清空路径记录表
  iter = iter + 1;
  Table = zeros(m,n);
end

%% 结果显示
[Shortest_Length,index] = min(Length_best);
Shortest_Route = Route_best(index,:);
disp(['最短距离:' num2str(Shortest_Length)]);
disp(['最短路径:' num2str([Shortest_Route Shortest_Route(1)])]);

%% 绘图
figure(1)
plot([citys(Shortest_Route,1);citys(Shortest_Route(1),1)],...
    [citys(Shortest_Route,2);citys(Shortest_Route(1),2)],'o-');
grid on
for i = 1:size(citys,1)
  text(citys(i,1),citys(i,2),[' ' num2str(i)]);
end
text(citys(Shortest_Route(1),1),citys(Shortest_Route(1),2),' 起点');
text(citys(Shortest_Route(end),1),citys(Shortest_Route(end),2),' 终点');
xlabel('城市位置横坐标')
ylabel('城市位置纵坐标')
title(['蚁群算法优化路径(最短距离:' num2str(Shortest_Length) ')'])
figure(2)
plot(1:iter_max,Length_best,'b',1:iter_max,Length_ave,'r:')
legend('最短距离','平均距离')
xlabel('迭代次数')
ylabel('距离')
title('各代最短距离与平均距离对比')
```